Month-End Review

Ayaan Khan | MiraFlores case study | Prepared for GreenGrowth CPAs

Prototype: https://ayaan-27.github.io/Miraflores_Project/

Offline prototype: Open OPEN_THIS.html from the submitted ZIP in a desktop browser. It needs no installation, server, account, or API key. A self-contained Python notebook with executed outputs is also included.

Source and instructions: https://github.com/ayaan-27/Miraflores_Project

1. The problems I considered

Understanding a financial movement across records. I have worked on text-to-SQL agents that retrieve information across relational databases. That motivated my interest in what happens after a number is retrieved: a reviewer still needs to explain the movement, find its support, and decide whether action is needed. I chose month-end expense variances as a concrete setting. An unexplained movement can lead to repeated queries, incomplete explanations, or a delayed close.

Connecting requests to useful evidence. At Ricoh, I worked on document retrieval that preserved references between service instructions and supporting diagrams. That motivated a second problem: matching an audit request to evidence covering the right entity, period, and requirement. Receiving a file does not establish that it resolves the request. Unsuitable support can create repeated follow-ups.

Preparing evaluation data without exposing sensitive information. At Mphasis, I worked on synthetic-data generation and sensitive-information redaction. Those projects motivated my interest in representative evaluation datasets. Restricted access can slow testing, while uncontrolled sharing creates privacy risks.

I selected variance review because it brings together structured data, supporting records, calculations, and a human decision in a small end-to-end workflow. It also demonstrates two useful outcomes: a movement may need an adjustment, or it may be legitimate. I have not interviewed GreenGrowth's staff or observed its close process. The frequency and cost of these problems at the firm remain assumptions, rather than research findings.

2. How I thought about the problem

My starting question was: what would let a reviewer move from “this account changed” to a defensible next step? A plausible explanation alone would not be enough. I wanted a traceable connection between the account movement, the underlying posting, its supporting evidence, the applicable policy, and the reviewer decision.

I assumed today's process involves comparing balances, drilling into a general ledger, locating invoices or agreements, and recording an explanation in a spreadsheet or close workpaper. I would validate that assumption before building integrations. The desired improvement is less repeated searching and clearer support for conclusions, while retaining reviewer control.

I reduced the first version to four stages: identify the movement, check its evidence, calculate a treatment, and request a decision. The important loop is missing evidence: the case should pause, retain the request, validate the response, and recompute before a new review. This made the state transitions and evidence checks more important than the choice of language model.

I initially explored a model-driven implementation. For the submitted version, I chose explicit rules and a self-contained browser app so a reviewer can immediately run and inspect the complete workflow. The notebook independently demonstrates the same accounting cases using Python's standard library. Neither submitted runtime requires a model or agent framework. A future model could help choose searches or draft explanations, but calculations and approval gates would remain in code.

A credible alternative is a spreadsheet with materiality formulas, saved ledger filters, and a standard checklist. That is my baseline. This prototype adds a connected evidence request, conditional treatment, decision history, and exportable workpaper. If those features do not reduce total reviewer effort, I would keep the spreadsheet or extend an existing platform.

The first scope is deliberately narrow: one positive posting explaining a material movement in a two-month expense close, using supplied structured evidence. The broader opportunity is an evidence-supported close workspace connected to accounting systems, if observation and evaluation establish demand.

3. The working workflow

1. Identify the movement. The app validates an expense-ledger CSV and calculates September and October balances. Movements meeting both an illustrative $5,000 and 20% threshold qualify for review. Duplicate transaction IDs, invalid amounts, unsupported currencies, and out-of-scope dates are rejected before replacing the workspace.

2. Check the evidence. The app compares the two periods and identifies an additional posting that reconciles the full movement. It checks posting uniqueness, entity, vendor, amount, currency, and relevant policy. An invoice without a service period triggers a request for its agreement. The request is a draft; no message is sent.

3. Calculate the treatment. A matching agreement must reference the expected invoice and contain the required service dates and authorization field. The app allocates the amount in integer cents under the supplied full-calendar-month policy, preserving the total when rounding. Unsupported partial months, earlier-period effects, multiple drivers, or mismatched sources require manual review. A proposed journal must balance.

4. Review and resume. A reviewer can request missing evidence, hold the case, or approve a supported workpaper. Approval is disabled while required checks are unresolved. Incorrect evidence leaves the pending case intact. A valid response resumes the same case, retains the earlier decision, recalculates, and requires a fresh decision. Browser storage retains progress on the same device; JSON export provides a portable review record. Reviewer names are entered, not authenticated.

Representative case. Software expense rises from $2,000 to $26,000. An additional $24,000 renewal explains the movement, but the invoice alone does not establish its service dates. The app requests the agreement. A mismatched agreement is rejected. The correct agreement covers October through the following September. Under the supplied policy, $2,000 belongs to October and $22,000 to later periods. The proposed journal debits prepaid services and credits software expense by $22,000. Adjusted October expense is $4,000, including the original recurring $2,000. The reviewer approves the workpaper; nothing is posted.

Other outcomes and outputs. An approved October bonus explains an $18,000 payroll movement without an adjustment. A current-month consulting agreement supports another movement without a deferral. The app exports a PDF workpaper, proposed-journal CSV, and JSON containing findings, sources, allocation, and decision history. The notebook includes executed examples and produces equivalent review data and a simple HTML workpaper.

All entities, transactions, policies, and evidence are synthetic. Evidence is supplied as structured JSON with sample quotes and page references; this release does not extract or authenticate uploaded PDFs. Source consistency is not proof of authenticity. Accounting-system connections, authenticated approval, immutable records, and production access controls are outside this prototype.

4. Validation, learning, and disclosure

The JavaScript controls and independently executable Python notebook check the expected $22,000 proposal and $4,000 adjusted expense, the no-adjustment cases, balanced journals, cent rounding, blocked approval, and rejection of unsuitable evidence and duplicate transaction IDs. Browser acceptance checks cover evidence upload and resumption, review persistence, invalid CSV preservation, exports, and mobile layout. These are small synthetic examples; they do not establish accuracy on unseen client records or accounting-policy suitability for GreenGrowth.

I would next observe three close reviewers using consented, redacted examples. I want to learn whether the bottleneck is retrieving transactions, locating support, interpreting treatment, or waiting for replies. I would compare the workflow with the spreadsheet baseline on five to ten willing datasets, measuring total review time, incorrect recommendations, unsuitable evidence assignments, and reopened cases. A provisional goal is 25% less reviewer time without increasing errors. This is a target to test, not an achieved saving.

I would simplify or stop if existing tools already solve the problem, preparing inputs takes more time than the workflow saves, or checking recommendations adds excessive effort. Validated demand would justify broader extraction, integrations, durable controlled storage, and testing across more accounting situations.

I used an AI coding assistant for problem framing, implementation, testing, and drafting. This version reuses accounting-control patterns and synthetic fixtures developed during earlier prototype iterations, plus the open-source jsPDF library for browser exports. No employer code or client data was used. Development and deployment experiments were iterative; I did not keep a reliable elapsed-time log and do not claim a measured completion time or a measured business benefit.